Phase 1 — Dev A Instructions

> **Your Role:** Project Setup & Base UI Components
> **Difficulty:** 🟢 Easy (Entry-level, learning project structure)
> **Timeline:** Week 1-2
> **Dependencies:** None (you start first!)
> **Total Tasks:** 18

🎯 Your Mission

You're setting up the **entire project foundation** and building the **core UI component library** that everyone else will use. Your work unblocks the entire team, so **quality over speed** is critical here.

📋 Task Checklist

Part 1: Project Bootstrapping (Day 1-2)

Task 0.1 — Create Vite Project

```
npm create vite@latest ahl-quran-web -- --template react-ts
cd ahl-quran-web
```

Task 0.2 — Install Core Dependencies

```
npm install react-router-dom @tanstack/react-query zustand axios
```

Task 0.3 — Install UI Dependencies

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
npm install clsx tailwind-merge
npm install lucide-react
```

Task 0.4 — Install Utility Dependencies

```
npm install react-hook-form zod @hookform/resolvers
```

```
npm install date-fns
```

Task 0.5 — Install Chart/Table Dependencies

```
npm install recharts @tanstack/react-table
```

Task 0.6 — Configure Tailwind with RTL Support

Create/update `tailwind.config.js`:

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {
      colors: {
        primary: '#0E9D6D',
        secondary: '#FF6B35',
      },
      fontFamily: {
        arabic: ['Cairo', 'sans-serif'],
      },
    },
  },
  plugins: [],
}
```

Add to `src/index.css`:

```
@import url('https://fonts.googleapis.com/css2?family=Cairo:wght@400;600;700;900&display=s

@tailwind base;
@tailwind components;
@tailwind utilities;

* {
  font-family: 'Cairo', sans-serif;
}

html[dir="rtl"] {
  direction: rtl;
}

html[dir="ltr"] {
  direction: ltr;
}
```

```
}

@layer base {
  body {
    @apply bg-gray-50 text-gray-900;
  }
}
```

Task 0.7 — Configure Vite with Path Aliases

Update `vite.config.ts`:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path'

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
})
```

Task 0.8 — Create Environment File

Create `.env.development`:

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
```

Task 0.9 — Configure TypeScript

Update `tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
```

```
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": ["src"],
  "references": [{ "path": "./tsconfig.node.json" }]
}
```

Task 0.10 — Create Folder Structure

```
mkdir -p src/{api,assets,components,hooks,layouts,pages,routes,store,types,utils}
mkdir -p src/components/ui
mkdir -p src/pages/{auth,dashboard,management,landing}
```

Create this structure:

```
src/
├── api/              # API service functions
├── assets/           # Images, fonts, etc.
├── components/       # Reusable components
│   └── ui/          # Base UI components (your focus!)
├── hooks/           # Custom React hooks
├── layouts/         # Layout components
├── pages/           # Page components
│   ├── auth/        # Login, Register
│   ├── dashboard/   # Role dashboards
│   ├── management/  # CRUD pages
│   └── landing/     # Public pages
├── routes/          # Route definitions
├── store/           # Zustand stores
├── types/           # TypeScript types
└── utils/           # Helper functions
```

Part 2: Base UI Components (Day 3-8)

Create each component in `src/components/ui/` directory.

Task 5.1.1 — Button Component

`src/components/ui/Button.tsx`:

```
import React from 'react';
import { cn } from '@/utils/cn';

interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: 'primary' | 'secondary' | 'outline' | 'danger' | 'ghost';
  size?: 'sm' | 'md' | 'lg';
  children: React.ReactNode;
}

export const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant = 'primary', size = 'md', children, ...props }, ref) => {
    const baseStyles = 'inline-flex items-center justify-center rounded-lg font-semibold t

    const variants = {
      primary: 'bg-primary text-white hover:bg-primary/90 focus:ring-primary',
      secondary: 'bg-secondary text-white hover:bg-secondary/90 focus:ring-secondary',
      outline: 'border-2 border-primary text-primary hover:bg-primary hover:text-white foc
      danger: 'bg-red-600 text-white hover:bg-red-700 focus:ring-red-600',
      ghost: 'text-gray-700 hover:bg-gray-100 focus:ring-gray-500',
    };

    const sizes = {
      sm: 'px-3 py-1.5 text-sm',
      md: 'px-4 py-2 text-base',
      lg: 'px-6 py-3 text-lg',
    };

    return (
      <button
        ref={ref}
        className={cn(baseStyles, variants[variant], sizes[size], className)}
        {...props}
      >
        {children}
      </button>
    );
  }
);

Button.displayName = 'Button';
```

Task 5.1.2 — Input Component

`src/components/ui/Input.tsx`:

```
import React from 'react';
import { cn } from '@/utils/cn';

interface InputProps extends React.InputHTMLAttributes<HTMLInputElement> {
  label?: string;
  error?: string;
```

```
  helperText?: string;
}

export const Input = React.forwardRef<HTMLInputElement, InputProps>(
  ({ className, label, error, helperText, id, ...props }, ref) => {
    const inputId = id || label?.toLowerCase().replace(/\s+/g, '-');

    return (
      <div className="w-full">
        {label && (
          <label
            htmlFor={inputId}
            className="block text-sm font-medium text-gray-700 mb-1"
          >
            {label}
          </label>
        )}
        <input
          id={inputId}
          ref={ref}
          className={cn(
            'w-full px-3 py-2 border rounded-lg text-gray-900 placeholder-gray-400',
            'focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent',
            'disabled:bg-gray-100 disabled:cursor-not-allowed',
            error ? 'border-red-500 focus:ring-red-500' : 'border-gray-300',
            className
          )}
          {...props}
        />
        {error && (
          <p className="mt-1 text-sm text-red-600">{error}</p>
        )}
        {helperText && !error && (
          <p className="mt-1 text-sm text-gray-500">{helperText}</p>
        )}
      </div>
    );
  }
);

Input.displayName = 'Input';
```

Task 5.1.3 — Select Component

`src/components/ui/Select.tsx`:

```
import React from 'react';
import { cn } from '@/utils/cn';

interface SelectProps extends React.SelectHTMLAttributes<HTMLSelectElement> {
  label?: string;
```

```
  label?: string;
  error?: string;
  options: Array<{ value: string | number; label: string }>;
}

export const Select = React.forwardRef<HTMLSelectElement, SelectProps>(
  ({ className, label, error, options, id, ...props }, ref) => {
    const selectId = id || label?.toLowerCase().replace(/\s+/g, '-');

    return (
      <div className="w-full">
        {label && (
          <label
            htmlFor={selectId}
            className="block text-sm font-medium text-gray-700 mb-1"
          >
            {label}
          </label>
        )}
        <select
          id={selectId}
          ref={ref}
          className={cn(
            'w-full px-3 py-2 border rounded-lg text-gray-900',
            'focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent',
            'disabled:bg-gray-100 disabled:cursor-not-allowed',
            error ? 'border-red-500 focus:ring-red-500' : 'border-gray-300',
            className
          )}
          {...props}
        >
          {options.map((option) => (
            <option key={option.value} value={option.value}>
              {option.label}
            </option>
          ))}
        </select>
        {error && (
          <p className="mt-1 text-sm text-red-600">{error}</p>
        )}
      </div>
    );
  }
);

Select.displayName = 'Select';
```

Task 5.1.4 — Modal Component

`src/components/ui/Modal.tsx`:

```tsx
import React, { useEffect } from 'react';
import { X } from 'lucide-react';
import { cn } from '@/utils/cn';

interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
  title?: string;
  children: React.ReactNode;
  footer?: React.ReactNode;
  size?: 'sm' | 'md' | 'lg' | 'xl';
}

export const Modal: React.FC<ModalProps> = ({
  isOpen,
  onClose,
  title,
  children,
  footer,
  size = 'md',
}) => {
  useEffect(() => {
    const handleEscape = (e: KeyboardEvent) => {
      if (e.key === 'Escape' && isOpen) {
        onClose();
      }
    };

    document.addEventListener('keydown', handleEscape);
    return () => document.removeEventListener('keydown', handleEscape);
  }, [isOpen, onClose]);

  useEffect(() => {
    if (isOpen) {
      document.body.style.overflow = 'hidden';
    } else {
      document.body.style.overflow = 'unset';
    }
    return () => {
      document.body.style.overflow = 'unset';
    };
  }, [isOpen]);

  if (!isOpen) return null;

  const sizes = {
    sm: 'max-w-md',
    md: 'max-w-lg',
    lg: 'max-w-2xl',
    xl: 'max-w-4xl',
  };

  return (
```

```
    <div className="fixed inset-0 z-50 overflow-y-auto">
      {/* Backdrop */}
      <div
        className="fixed inset-0 bg-black bg-opacity-50 transition-opacity"
        onClick={onClose}
      />

      {/* Modal */}
      <div className="flex min-h-full items-center justify-center p-4">
        <div
          className={cn(
            'relative w-full bg-white rounded-lg shadow-xl',
            sizes[size]
          )}
          onClick={(e) => e.stopPropagation()}
        >
          {/* Header */}
          {title && (
            <div className="flex items-center justify-between p-6 border-b">
              <h2 className="text-xl font-bold text-gray-900">{title}</h2>
              <button
                onClick={onClose}
                className="text-gray-400 hover:text-gray-600 transition-colors"
              >
                <X size={24} />
              </button>
            </div>
          )}

          {/* Body */}
          <div className="p-6">{children}</div>

          {/* Footer */}
          {footer && (
            <div className="flex items-center justify-end gap-3 p-6 border-t bg-gray-50">
              {footer}
            </div>
          )}
        </div>
      </div>
    </div>
  );
};
```

Task 5.1.5 — ConfirmDialog Component

`src/components/ui/ConfirmDialog.tsx`:

```
import React from 'react';
import { Modal } from './Modal';
import { Button } from './Button';
```

```tsx
interface ConfirmDialogProps {
  isOpen: boolean;
  onClose: () => void;
  onConfirm: () => void;
  title: string;
  message: string;
  confirmText?: string;
  cancelText?: string;
  variant?: 'danger' | 'primary';
}

export const ConfirmDialog: React.FC<ConfirmDialogProps> = ({
  isOpen,
  onClose,
  onConfirm,
  title,
  message,
  confirmText = 'تأكيد',
  cancelText = 'إلغاء',
  variant = 'primary',
}) => {
  const handleConfirm = () => {
    onConfirm();
    onClose();
  };

  return (
    <Modal
      isOpen={isOpen}
      onClose={onClose}
      title={title}
      size="sm"
      footer={
        <>
          <Button variant="ghost" onClick={onClose}>
            {cancelText}
          </Button>
          <Button variant={variant} onClick={handleConfirm}>
            {confirmText}
          </Button>
        </>
      }
    >
      <p className="text-gray-700">{message}</p>
    </Modal>
  );
};
```

Task 5.1.6 — Toast Component

`src/components/ui/Toast.tsx` :

```tsx
import React, { useEffect } from 'react';
import { CheckCircle, XCircle, Info, X } from 'lucide-react';
import { cn } from '@/utils/cn';

interface ToastProps {
  type: 'success' | 'error' | 'info';
  message: string;
  isVisible: boolean;
  onClose: () => void;
  duration?: number;
}

export const Toast: React.FC<ToastProps> = ({
  type,
  message,
  isVisible,
  onClose,
  duration = 5000,
}) => {
  useEffect(() => {
    if (isVisible && duration > 0) {
      const timer = setTimeout(() => {
        onClose();
      }, duration);

      return () => clearTimeout(timer);
    }
  }, [isVisible, duration, onClose]);

  if (!isVisible) return null;

  const icons = {
    success: <CheckCircle className="text-green-600" size={24} />,
    error: <XCircle className="text-red-600" size={24} />,
    info: <Info className="text-blue-600" size={24} />,
  };

  const styles = {
    success: 'bg-green-50 border-green-200',
    error: 'bg-red-50 border-red-200',
    info: 'bg-blue-50 border-blue-200',
  };

  return (
    <div className="fixed top-4 right-4 z-50 animate-in slide-in-from-top">
      <div
        className={cn(
          'flex items-center gap-3 p-4 rounded-lg border shadow-lg min-w-[300px]',
          styles[type]
        )}
      >
        {icons[type]}
```

```
        {icons[type]}
        <p className="flex-1 text-gray-900 font-medium">{message}</p>
        <button
          onClick={onClose}
          className="text-gray-500 hover:text-gray-700 transition-colors"
        >
          <X size={20} />
        </button>
      </div>
    </div>
  );
};
```

Task 5.1.7 — Spinner Component

`src/components/ui/Spinner.tsx`:

```
import React from 'react';
import { cn } from '@/utils/cn';

interface SpinnerProps {
  size?: 'sm' | 'md' | 'lg';
  className?: string;
}

export const Spinner: React.FC<SpinnerProps> = ({ size = 'md', className }) => {
  const sizes = {
    sm: 'w-4 h-4 border-2',
    md: 'w-8 h-8 border-3',
    lg: 'w-12 h-12 border-4',
  };

  return (
    <div
      className={cn(
        'inline-block animate-spin rounded-full border-solid border-primary border-r-trans
        sizes[size],
        className
      )}
      role="status"
      aria-label="التحاويلي"
    >
      <span className="sr-only">التحاويلي...</     span>
    </div>
  );
};
```

Task 5.1.8 — Badge Component

`src/components/ui/Badge.tsx`:

```
import React from 'react';
import { cn } from '@/utils/cn';

interface BadgeProps {
  children: React.ReactNode;
  variant?: 'success' | 'warning' | 'danger' | 'info' | 'default';
  className?: string;
}

export const Badge: React.FC<BadgeProps> = ({
  children,
  variant = 'default',
  className,
}) => {
  const variants = {
    success: 'bg-green-100 text-green-800 border-green-200',
    warning: 'bg-yellow-100 text-yellow-800 border-yellow-200',
    danger: 'bg-red-100 text-red-800 border-red-200',
    info: 'bg-blue-100 text-blue-800 border-blue-200',
    default: 'bg-gray-100 text-gray-800 border-gray-200',
  };

  return (
    <span
      className={cn(
        'inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-semibold border'
        variants[variant],
        className
      )}
    >
      {children}
    </span>
  );
};
```

Task 5.1.9 — Avatar Component

`src/components/ui/Avatar.tsx`:

```
import React from 'react';
import { User } from 'lucide-react';
import { cn } from '@/utils/cn';

interface AvatarProps {
  src?: string;
  alt?: string;
  name?: string;
  size?: 'sm' | 'md' | 'lg';
  className?: string;
```

```
}

export const Avatar: React.FC<AvatarProps> = ({
  src,
  alt,
  name,
  size = 'md',
  className,
}) => {
  const sizes = {
    sm: 'w-8 h-8 text-xs',
    md: 'w-10 h-10 text-sm',
    lg: 'w-12 h-12 text-base',
  };

  const getInitials = (name?: string) => {
    if (!name) return '';
    const parts = name.split(' ');
    return parts
      .slice(0, 2)
      .map((part) => part[0])
      .join('')
      .toUpperCase();
  };

  return (
    <div
      className={cn(
        'inline-flex items-center justify-center rounded-full overflow-hidden bg-primary t
        sizes[size],
        className
      )}
    >
      {src ? (
        <img src={src} alt={alt || name} className="w-full h-full object-cover" />
      ) : (
        <span>{getInitials(name) || <User size={size === 'sm' ? 16 : 20} />}</span>
      )}
    </div>
  );
};
```

Task 5.1.10 — Tooltip Component

`src/components/ui/Tooltip.tsx`:

```
import React, { useState } from 'react';
import { cn } from '@/utils/cn';

interface TooltipProps {
  children: React.ReactElement;
```

```
  content: string;
  position?: 'top' | 'bottom' | 'left' | 'right';
}

export const Tooltip: React.FC<TooltipProps> = ({
  children,
  content,
  position = 'top',
}) => {
  const [isVisible, setIsVisible] = useState(false);

  const positions = {
    top: 'bottom-full left-1/2 -translate-x-1/2 mb-2',
    bottom: 'top-full left-1/2 -translate-x-1/2 mt-2',
    left: 'right-full top-1/2 -translate-y-1/2 mr-2',
    right: 'left-full top-1/2 -translate-y-1/2 ml-2',
  };

  return (
    <div className="relative inline-block">
      {React.cloneElement(children, {
        onMouseEnter: () => setIsVisible(true),
        onMouseLeave: () => setIsVisible(false),
      })}
      {isVisible && (
        <div
          className={cn(
            'absolute z-50 px-2 py-1 text-xs text-white bg-gray-900 rounded shadow-lg whit
            positions[position]
          )}
        >
          {content}
        </div>
      )}
    </div>
  );
};
```

Part 3: Utility Function (Day 9)

Task 21.1 — Create cn utility

`src/utils/cn.ts`:

```
import { clsx, type ClassValue } from 'clsx';
import { twMerge } from 'tailwind-merge';

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
```

```
}
```

✅ Definition of Done

Before marking Phase 1 complete, verify:

- ☐ All dependencies installed without errors
- ☐ `npm run dev` starts the development server successfully
- ☐ All 10 UI components render correctly
- ☐ Components are properly typed (no TypeScript errors)
- ☐ Components are documented with clear props interfaces
- ☐ All components use the `cn` utility for className merging
- ☐ Tailwind styles are working (test by adding `className="bg-primary"` to a div)
- ☐ Path alias `@/` works (test by importing a component with `@/components/ui/Button`)

🧪 Testing Your Work

Create a test page to verify all components work:

`src/pages/ComponentShowcase.tsx`:

```
import { Button } from '@/components/ui/Button';
import { Input } from '@/components/ui/Input';
import { Select } from '@/components/ui/Select';
import { Badge } from '@/components/ui/Badge';
import { Avatar } from '@/components/ui/Avatar';
import { Spinner } from '@/components/ui/Spinner';

export const ComponentShowcase = () => {
  return (
    <div className="p-8 space-y-8" dir="rtl">
      <h1 className="text-3xl font-bold">المكونات</     h1>

      <section className="space-y-4">
        <h2 className="text-xl font-bold">الأزرار</h2>
        <div className="flex gap-2">
          <Button variant="primary">أساسي</Button>
          <Button variant="secondary">ثانوي</Button>
          <Button variant="outline">حدود</Button>
          <Button variant="danger">خطر</Button>
        </div>
      </section>
```

```
      <section className="space-y-4">
        <h2 className="text-xl font-bold">الحقول</h2>
        <Input label="الإلكتروني"     placeholder="بريدك الإلكتروني"        >
        <Input label="كلمة المرور"    type="password" error="كلمة المرور مطلوبة"        >
        <Select
          label="الدور"
          options={[
            { value: 'president', label: 'رئيس' },
            { value: 'teacher', label: 'معلم' },
          ]}
        />
      </section>

      <section className="space-y-4">
        <h2 className="text-xl font-bold">الشارات</h2>
        <div className="flex gap-2">
          <Badge variant="success">نشط</Badge>
          <Badge variant="danger">نشيط</   Badge>
          <Badge variant="warning">معلق</Badge>
        </div>
      </section>

      <section className="space-y-4">
        <h2 className="text-xl font-bold">الصورة الرمزية</     h2>
        <div className="flex gap-2">
          <Avatar name="أحمد"     size="sm" />
          <Avatar name="فاطمة"     size="md" />
          <Avatar name="محمد"     size="lg" />
        </div>
      </section>

      <section className="space-y-4">
        <h2 className="text-xl font-bold">المحمل</h2>
        <div className="flex gap-4">
          <Spinner size="sm" />
          <Spinner size="md" />
          <Spinner size="lg" />
        </div>
      </section>
    </div>
  );
};
```

Add this to your routes temporarily to test.

💡 Tips for Success

1. **Don't rush** — Your work is the foundation for everyone else
2. **Test each component** — Make sure it works before moving to the next

3. **Follow naming conventions** — Use PascalCase for components, camelCase for functions
4. **Add comments** — Explain complex logic
5. **Ask for help** — If stuck on Tailwind or TypeScript, ask Dev B or C

🚀 What's Next?

After completing Phase 1, you'll move to **Phase 2: Student Management** where you'll build a complete CRUD system using the components you just created!

📞 Need Help?

- **Tailwind questions:** Check docs at tailwindcss.com
- **Component design questions:** Ask Dev F (they're building landing pages)
- **TypeScript errors:** Ask Dev B (they're the types expert)
- **Blockers:** Post in #react-rebuild-blockers Slack channel immediately

Good luck! 🎉